# 40.2 ACP 会话管理与运行时

> **生成模型**：GPT-5.4（openai/gpt-5.4）
> **Token 消耗**：输入 ~22k tokens，输出 ~3.4k tokens（本节，估算）

32.1 解决的是“怎么把 ACP 请求接进来”，32.2 要看的是另一个问题：请求进来以后，会话怎么落地、怎么恢复、怎么在运行时里保持一致。OpenClaw 在这里没有搞一套特别夸张的中间层，而是把问题拆成几块：轻量会话存储、会话键解析、会话恢复、身份追踪、元数据持久化，以及运行时缓存。这样一来，ACP 前端看到的是一个连续会话，后端其实在做几层映射和协调。

## 40.2.1 会话存储（AcpSessionStore）

ACP 适配层先要维护自己的会话表。 `src/acp/session.ts` 里的 **会话存储**（AcpSessionStore）就是这层最靠前的状态容器：它不负责永久保存历史，只负责让 ACP 连接在一次进程生命周期里能找到对应的会话、活跃运行和工作目录。

源码里的会话记录很小，字段也很克制：

```
// src/acp/session.ts
type AcpSession = {
  sessionId: SessionId;
  sessionKey: string;
  cwd: string;
  createdAt: number;
  lastTouchedAt: number;
  abortController: AbortController | null;
  activeRunId: string | null;
};
// Max 5000 sessions, 24h idle TTL, eviction on access
```

这里的实现本质上是一个内存 **映射表**（Map）。 `sessionId -> AcpSession` 是主索引， `runId -> sessionId` 是反向索引，后者让运行中的请求能快速回查到所属会话。 `lastTouchedAt` 每次访问都会更新，所以它同时承担了“最近使用时间”的角色。

更有意思的是它的驱逐策略。OpenClaw 默认最多保留 5000 个会话，空闲超时时间是 24 小时，而且不是靠后台定时器清理，而是在创建和访问路径上顺手执行回收。这样做实现很简单，也避免了额外的定时任务噪声；代价是清理时机不是严格实时，但对 ACP 这种桥接层来说已经够用了。

`abortController` 和 `activeRunId` 则把“会话”与“当前这次运行”挂在一起。新 prompt 进来时，如果旧运行还活着，就可以通过 `cancelActiveRun()` 中断旧请求。这意味着 ACP 侧的一个会话在同一时刻只允许一个活跃运行，模型简单，行为也更容易预测。

> **衍生解释：TTL 驱逐（Time-To-Live Eviction）**
>
> TTL 就是“过期时间”。很多系统会给缓存对象设置一个最大空闲时长，超过这个时长就认为它不值得继续留在内存里。这里的 TTL 不是消息历史的保留期，而是 ACP 进程内临时会话对象的空闲寿命。

## 40.2.2 会话键解析（Session Key Resolution）

ACP 客户端给的 `sessionId` ，并不等于 OpenClaw 内部真正使用的 `sessionKey` 。两者之间要经过一次解析，逻辑放在 `src/acp/session-mapper.ts` 。这一步看着像参数整理，其实决定了“我要接到哪个后端会话上”。

解析顺序一共五步，优先级很明确：

1. 先看显式 `sessionLabel` ，调用 `gateway.request("sessions.resolve", { label })`
2. 再看显式 `sessionKey` ，如果 `requireExisting` 为真，就先校验是否存在
3. 然后看启动参数里的 `defaultSessionLabel`
4. 再看启动参数里的 `defaultSessionKey`
5. 最后才退回到 `acp:{sessionId}`

可以把它理解成“先听用户点名，再听默认配置，最后自己起一个临时名字”。源码结构基本就是这样：

```
export async function resolveSessionKey(params: {
  meta: AcpSessionMeta;
  fallbackKey: string;
  gateway: GatewayClient;
  opts: AcpServerOptions;
}): Promise<string> {
  const requestedLabel = params.meta.sessionLabel ?? params.opts.defaultSessionLabel;
  const requestedKey = params.meta.sessionKey ?? params.opts.defaultSessionKey;

  if (params.meta.sessionLabel) {
    return (await params.gateway.request("sessions.resolve", {
      label: params.meta.sessionLabel,
    })).key;
  }

  // ...显式 key、默认 label、默认 key 的分支...
  return params.fallbackKey;
}
```

这里有个设计细节很实用： `label` 和 `key` 都支持“显式传入”和“启动默认值”两层来源，但显式值总是更强。这保证了 IDE 插件既能配置一个长期默认会话，也能在某次连接里强制切换到另一个会话，不会互相打架。

## 40.2.3 会话恢复（Session Resume）

如果用户不是新开一段对话，而是想接着之前的 ACP 运行继续，运行时就要支持恢复。这个入口出现在 `runtime.ensureSession()` ：控制面在初始化会话时可以把 `resumeSessionId` 一并传进去。

`manager.core.ts` 里对应的调用很直接：

```
await runtime.ensureSession({
  sessionKey,
  agent,
  mode: input.mode,
  resumeSessionId: input.resumeSessionId,
  cwd: requestedCwd,
});
```

不过，恢复不是只靠一个参数就结束了。ACP 客户端重新加载会话时，还要把旧 transcript 补回前端视图。 `translator.ts` 的 `loadSession()` 会同时拉取会话快照和消息转录，再通过 `sessionUpdate`

事件逐条重放给客户端。

```
const ACP_LOAD_SESSION_REPLAY_LIMIT = 1_000_000;

private async getSessionTranscript(sessionKey: string) {
  return await this.gateway.request("sessions.get", {
    key: sessionKey,
    limit: ACP_LOAD_SESSION_REPLAY_LIMIT,
  });
}

private async replaySessionTranscript(sessionId: string, transcript: GatewayTranscriptM
  for (const message of transcript) {
    await this.connection.sessionUpdate({
      sessionId,
      update: {
        sessionUpdate: role === "user" ? "user_message_chunk" : "agent_message_chunk",
        content: { type: "text", text },
      },
    });
  }
}
```

这里的上限是 100 万条消息，数值很大，说明它更像一个“保护阈值”而不是日常推荐规模。恢复时采用事件重放，而不是一次性塞给客户端一个大数组，原因也很清楚：ACP 客户端原本就围绕 `sessionUpdate` 消费数据，重放老消息和接收新消息可以共用同一条渲染路径。

## 40.2.4 会话身份追踪（Session Identity）

会话恢复真正麻烦的地方，不是文本历史，而是“这个会话在不同层里到底叫什么”。OpenClaw 在 `src/acp/runtime/session-identity.ts` 里单独做了一套 **会话身份**（Session Identity）追踪，用来连接 ACP 控制面、后端运行时，以及上游 agent harness 的不同标识。

核心状态和来源如下：

```
// src/acp/runtime/session-identity.ts
// States: "pending" | "resolved"
// Sources: "ensure" | "status" | "event"
// Fields: acpxRecordId, acpxSessionId, agentSessionId
// Merging: incoming updates current, preserves resolved state
```

这几个字段别混了。 `acpxRecordId` 更像后端内部记录号， `acpxSessionId` 是后端 ACP 运行时的会话标识， `agentSessionId` 则可能来自更上游的 agent 会话系统。OpenClaw 没假设它们会同时出现，也没假设第一次 `ensure` 就一定拿得到全部信息，所以身份对象允许先处在 `pending` 状态。

真正关键的是合并策略。 `mergeSessionIdentity()` 并不是“新值来了就全量覆盖”，而是偏保守：如果当前身份已经 `resolved` ，后续一个信息更少的更新不能把它降回去；只有更完整、同样 resolved 的更新才能替换相关字段。这个策略避免了事件乱序或状态回退把已经确定的会话 ID 冲掉。

> **衍生解释：最终一致性（Eventual Consistency）**
>
> 有些分布式系统里，不同组件不会在同一时刻拿到同一份最新状态，而是允许短时间“不一致”，最后再慢慢收敛。这里的会话身份就有点这个味道： `ensure` 、 `status` 、 `event` 可能先后带来不同粒度的信息，系统要做的是把它们并起来，而不是要求第一拍就完整。

## 40.2.5 会话元数据持久化

前面的 `AcpSessionStore` 只存在于内存里，进程一停就没了。真正跨进程保留 ACP 会话状态的，是 `src/acp/runtime/session-meta.ts` 对 `sessions.json` 的读写。它把 ACP 相关元数据嵌回 OpenClaw 现有的会话存储里，而不是另起一套数据库。

元数据结构是这样的：

```
// src/acp/runtime/session-meta.ts - stored in sessions.json
type SessionAcpMeta = {
  backend: string;
  agent: string;
  runtimeSessionName: string;
  identity?: SessionAcpIdentity;
  mode: "persistent" | "oneshot";
  runtimeOptions?: AcpSessionRuntimeOptions;
  state: "idle" | "running" | "error";
  lastActivityAt: number;
  lastError?: string;
};
```

这份数据的价值在于：下一次再解析到同一个 `sessionKey` 时，系统可以知道它上次绑定的是哪个 backend、哪个 agent、什么模式、最近有没有出错，还能拿到上文讲的 identity。换句话说，ACP 会话不只是“还有没有聊天记录”，而是“这条会话在运行时语义上长什么样”。

`session-meta.ts` 里的实现也比较稳。它会先根据 `sessionKey` 找到对应 `sessions.json` 路径，再做大小写兼容查找，最后通过 `updateSessionStore()` 原子更新条目。这样做能减少并发写入时把整

个会话文件搞坏的风险，也能兼容历史上键名大小写不一致的情况。

## 40.2.6 运行时缓存与驱逐

到了控制面这一层，真正昂贵的对象不是那张小小的会话表，而是 ACP **运行时实例**（Runtime Instance）。这些对象可能持有后端连接、会话句柄、能力信息，所以 OpenClaw 在 `src/acp/control-plane/runtime-cache.ts` 做了一层缓存。

`RuntimeCache` 的风格很像简化版 LRU。每个条目都带 `lastTouchedAt` ， `get()` 默认会 touch， `collectIdleCandidates()` 按空闲时长收集可驱逐对象。它没有维护严格的链表顺序，但行为上已经具备“最近访问的别急着删，长时间不用的优先淘汰”的特征。

真正的驱逐动作在 `manager.core.ts` 里完成：系统会根据配置算出空闲 TTL，扫描缓存里的候选项，然后对每个候选会话加串行锁、再次确认没有活跃 turn，最后调用运行时的 `close({ reason: "idle-evicted" })` 。这一步不是单纯删 Map，而是显式通知后端释放资源。

另一个经常被忽略的点是 **每会话 actor 队列**（Per-session Actor Queue）。 `src/acp/control-plane/session-actor-queue.ts` 用 `KeyedAsyncQueue` 按 `actorKey` 串行化操作，同一会话上的初始化、状态读取、turn 执行、驱逐不会并发踩来踩去。对于会话系统来说，这种“同 key 串行、不同 key 并行”的设计很常见，因为它正好对应“单会话状态机”的需求。

```
export class SessionActorQueue {
  private readonly queue = new KeyedAsyncQueue();

  async run<T>(actorKey: string, op: () => Promise<T>): Promise<T> {
    return this.queue.enqueue(actorKey, op, {
      onEnqueue: () => {
        // 统计该会话待处理任务数
      },
      onSettle: () => {
        // 任务结束后递减计数
      },
    });
  }
}
```

如果把这一节几部分连起来看，你会发现 OpenClaw 的 ACP 运行时管理思路其实很务实：前台靠轻量内存表接住 ACP 连接，后台靠 `sessions.json` 留住跨进程语义，中间再用身份追踪和串行队列把状态拼稳。这样既没有把 ACP 写成“纯临时桥”，也没有为了桥接协议引入一整套沉重的新存储系统。

# 本节小结

1. `AcpSessionStore` 是进程内会话表，基于 Map，带 5000 上限、24 小时空闲 TTL、访问时触发回收。
2. 会话键解析有固定五步优先级：显式 label、显式 key、默认 label、默认 key、最后回退到 `acp:{sessionId}` 。
3. 会话恢复分成两层：运行时通过 `resumeSessionId` 续接，ACP 前端通过 transcript 重放恢复可见历史。
4. `Session Identity` 用 `pending/resolved` 与 `ensure/status/event` 三类来源追踪多层会话 ID，并通过保守合并避免状态回退。
5. ACP 元数据持久化在 `sessions.json` 中，保存 backend、agent、mode、runtimeOptions、identity 和最近状态。
6. 运行时缓存负责复用昂贵句柄，空闲驱逐负责释放资源，而每会话 actor 队列保证同一会话上的操作串行执行。